

“Tom’s House” Out of Order Processor Final Report

Introduction & Overview

Modern high-performance processors rely on out-of-order execution and speculative techniques to improve instruction throughput while maintaining correct program behavior. The goal of this project was to design and implement a fully functional out-of-order RISC-V processor, providing us hand-on experience on its microarchitecture to enable high instruction-level parallelism in real CPUs. RISC-V is an open-source instruction set architecture that defines how software instructions are encoded and executed, making it an ideal platform for exploring processor design without proprietary constraints.

This processor was developed incrementally across three checkpoints, beginning with a basic instruction fetch pipeline and culminating in a Tomasulo-style core with register renaming, reservation stations, and in-order retirement using a reorder buffer. Each checkpoint added architectural complexity while preserving correctness, allowing performance tradeoffs to be explored in a controlled manner. Advanced features such as branch prediction, prefetching, and return address optimization were implemented to reduce pipeline stalls and improve instruction throughput relative to simpler baseline designs.

The remainder of this report is organized as follows: the first sections describe the design and verification of each checkpoint, our infrastructure design decisions, followed by detailed explanations of the advanced performance optimizations. The report concludes with a discussion of design tradeoffs, performance impact, and lessons learned from building a realistic out-of-order processor.

Infrastructure

For this project, we implemented a Tomasulo-style processor, however we had a separate RAT and physical register file, decoupled from the ROB. We chose this infrastructure because it cleanly separates register renaming from in-order retirement, simplifies dependency tracking

across speculative execution, and enables more aggressive out-of-order issue without overloading the ROB with rename state logic .

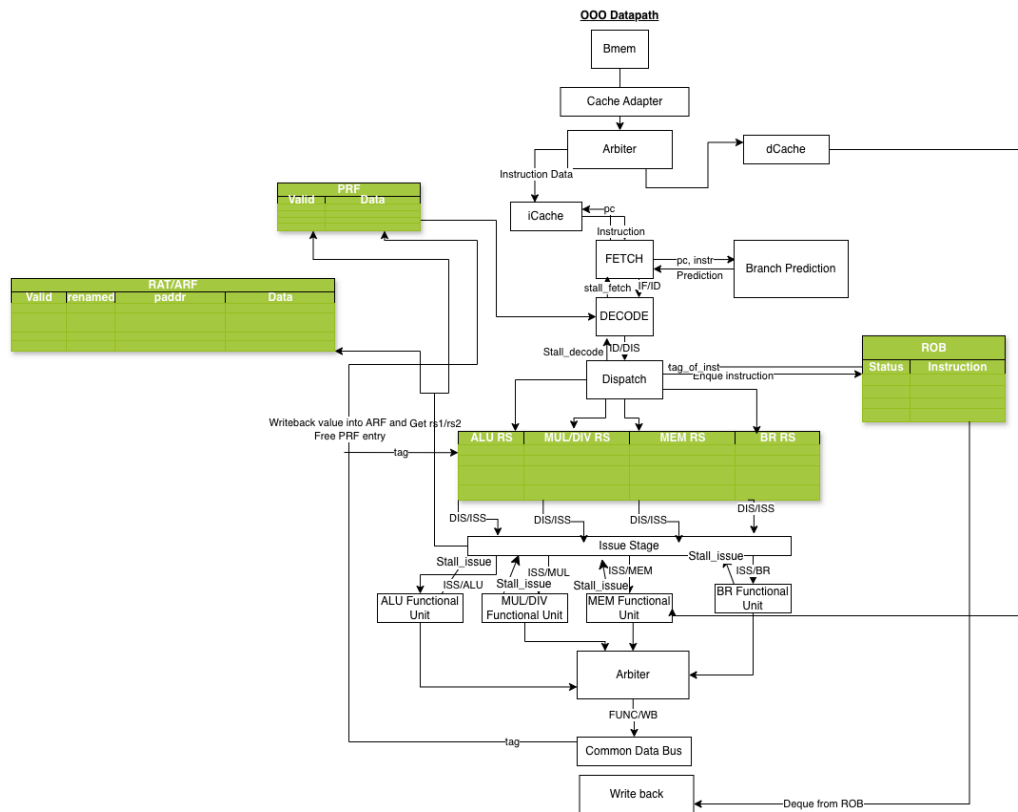


Figure 1: High-level datapath of the out-of-order processor

Milestones

Checkpoint 1

Our checkpoint 1 design involved a fetch stage, which was equipped with an instruction cache with a line buffer, an instruction FIFO, and a cacheline adapter. We scripted separate test-benches for the cacheline adapter and FIFO modules. Both modules were very important, since the FIFO was used in many other areas, and the cacheline adapter was eventually used for the data cache in checkpoint 3.

To test our fetch stage, we simply hooked up our instruction FIFO output to the Spike RVFI interface with a specific set of instructions. These instructions were simply ALU instructions that

only used register x0, such that all source values and return values would be 0. That way, we only needed to hook up the actual instruction word, order, and valid to the RVFI interface.

Using the line buffer, this implementation was able to have zero latency between instructions in the same cacheline. However, to test that we could actually fetch from the cache itself, we temporarily removed the line buffer to make sure the cache returned the correct data. This resulted in having 1-cycle gaps between fetches, so we obviously re-installed the line buffer.

Checkpoint 2

For checkpoint 2, we were tasked with implementing ALU instructions and Multiply/Divide instructions using Synopsis IPs. For out-of-order execution, we needed to create the entire shell of a Tomasulo processor, such as the ROB, register renaming, and reservation stations.

Our register renaming scheme was unique, since it wasn't necessarily Tomasulo's or ERR since we incorrectly believed the lecture slides were teaching Tomasulo's. This scheme involved a combined RAT and ARF, where ARF contains committed data. Registers were renamed on dispatch, and reservation station entries read from the PRF to determine when to wake up. We used a free-list FIFO to determine available PRF entries, but, during advanced features, was replaced with tying the PRF index straight to the ROB entry for simplicity.

For the Mul/Div unit, we initially had 3 stage sequential multiply and 12-stage divide. These weren't particularly hard to implement. Our strategy for dealing with signed and unsigned inputs was to sign-extend signed inputs by 1 bit and zero-extend unsigned inputs by 1 bit. To test our Mul/Div implementation, we initiated registers with signed and unsigned values, then tested all forms of multiply and divide with each combination of them. That way, we were able to test how our multiplier dealt with signed and unsigned values as well as general correctness. We performed similar tests for our ALU instructions.

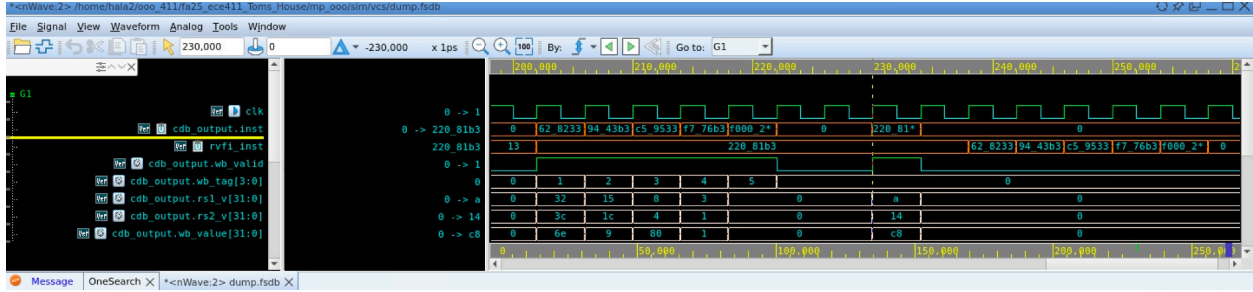


Figure 2: Waveform depicting OOO execution after CP2.

Checkpoint 3

For Checkpoint 3, we focused on functional completeness by adding support for memory operations, branches, jumps, and AUIPC, as well as integrating the data cache and arbitration logic. Our main goal was to get all instructions executing correctly rather than optimizing performance.

Firstly, we encountered multiple issues in the memory functional unit, including incorrect stalling behavior, missing backpressure, and inconsistent use of aligned addresses when interfacing with the data cache. These bugs caused loads and stores to either block unnecessarily or access incorrect memory locations. We also saw some timing issues around inflight memory requests, where responses were not always cleanly matched to the correct instruction.

One of the other bugs we encountered was related to branch recovery. When first implementing branching, we forgot to correctly integrate flush handling in several parts of the pipeline, most notably in the RAT and physical register file. As a result, on a branch misprediction or redirect, speculative rename state was not fully cleared, which led to incorrect register mappings being reused after the flush.

By the end of the checkpoint, we resolved these issues by auditing flush propagation across all speculative structures, fixing address alignment, and tightening valid/ready logic in the memory pipeline.

Advanced Features

For the final checkpoint in our out of order design, we were tasked with optimizing our processor such that it hits the required IPC, area, and PD⁴ requirements. This involved adding better branch prediction to prevent costly pipeline flushes, reducing the size of certain modules to fit under area requirements, improving our cache's AMAT through prefetching, and implementing a split LSQ to allow loads and stores to execute more efficiently.

For the branch predictor, we implemented a BTB-based Gshare branch predictor. The biggest issue we faced was improving the accuracy of the branch predictor while keeping the area of it as low as possible. This was done with better index hashing, using a global PHT over a local PHT, and reducing the size of the data structures used. We were able to create an effective enough predictor that was area and time efficient through the strategies listed before.

To make loads more efficient, we implemented a split LSQ to allow loads and stores to execute out of order rather than having to wait until they reached the top of the ROB. This significantly increased performance on memory intensive programs.

To lower our overall AMAT on our ICache and DCache, we implemented two forms of prefetching. On our ICache, we implemented a basic next line prefetching function. This made sense to implement since the typical flow of most programs is to just fetch instructions that are next to each other in memory. On our DCache, we implemented a stride prefetcher since most of the benchmarks tested on our processor accessed memory locations close to one another frequently.

Our biggest struggle, however, was reducing the critical path of our design to hit the PD⁴ benchmarks set by the class. Our maximum clock frequency was too low to come close to beating the benchmarks outlined by the course staff due to our large critical path in between our ROB, split LSQ, and Dcache access.

Gshare Branch predictor with Branch Target Buffer

To decrease control hazard penalties during the fetch stage, a Branch Target Buffer (BTB) global history branch predictor with a Gshare pattern history table (PHT) was used. In a BTB hit where the recovered tag matches, a 2-bit saturating counter in the PHT determines whether the branch is predicted taken or not. The PHT index is calculated by XORing a segment of the PC with a global history register (GHR). A PHT entry indicates if, given correlations in recent branch outcomes and current control flow, the branch should be predicted as being taken or not. If the branch is predicted to be taken, the PC is immediately updated to the target address. Otherwise, execution continues with the fall-through address ($PC + 4$).

The predictor learns solely based on the committed branch outcomes it gets from the ROB in order to prevent corrupting predictor state due to speculation. Based on a commit, a copy of the BTB table entry is updated with a resolved target address, a PHT counter is incremented (if branch taken) or decremented (if branch not taken), and the GHR is shifted in with the resolved taken/not-taken bit. This design helps reduce pipeline flushes on loop-intensive and control-intensive code, resulting in an increased IPC while not sacrificing area. While the direct-mapped BTB and shared PHT introduce some aliasing, the global-history XOR indexing mitigates most conflicts and provides better accuracy than a simple two-bit predictor.

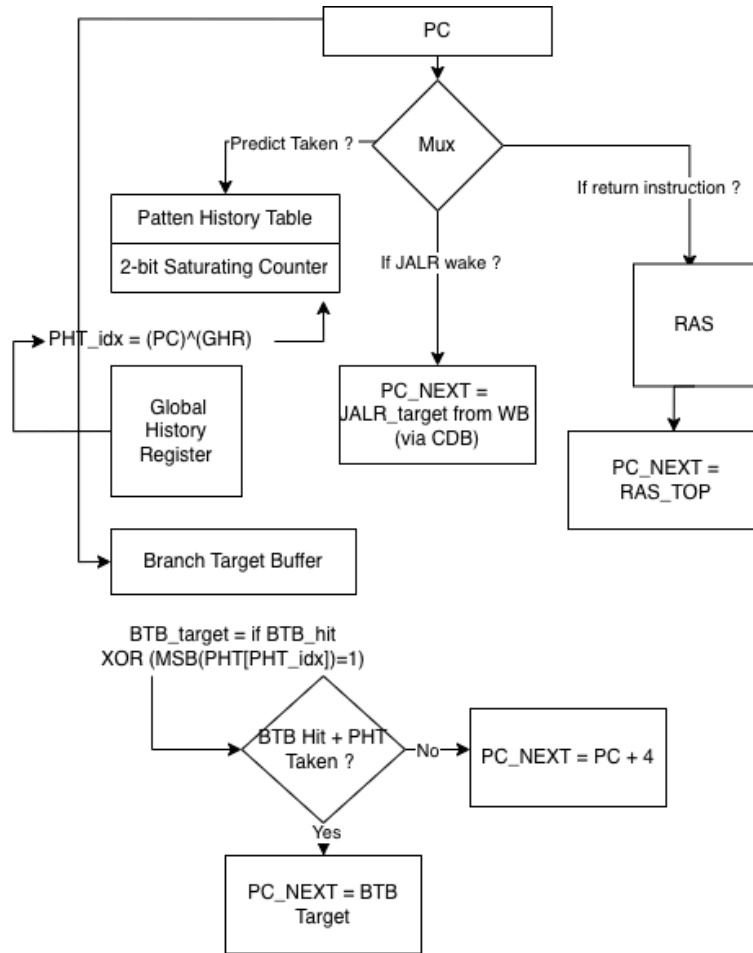


Figure 3: Flowchart depicting high-level branch prediction logic.

Below is a table that shows IPC statistics when using this branch predictor vs when doing a static “not taken” branch prediction on the major benchmarks tested. The branch predictor’s parameters were set to 32 BHT entries, 32 PHT entries, and used a 5 bit GHR.

Benchmark	IPC (No Branch Pred)	IPC (With Branch Pred)
Coremark	0.393239	0.5023
AES_SHA	0.465294	0.4691
fft	0.549612	0.59
Compression	0.464641	0.5994
Mergesort	0.452682	0.4859
*clock set to 250 MHz for all tests		

Figure 4: IPC improvements after integrating branch prediction.

Split LSQ

The memory functional unit is designed around two queues. The module maintains explicit Load and Store Queues as its primary infrastructure, and uses a simple scan loop for load allocation, a circular FIFO for stores, and backpressure mechanisms to prevent structural hazards.

Address generation is performed once operands become ready in the reservation station. At that point, the unit computes aligned addresses, byte offsets, and access masks, and records them into the appropriate queue entry, decoupling address resolution from cache access. Issue control is intentionally meant to be conservative, only a single memory operation may be inflight at a time. Stores are always prioritized, while loads may issue out of order only when no older store can legally block them. This is enforced using relative age inferred from ROB tag distance to the commit pointer.

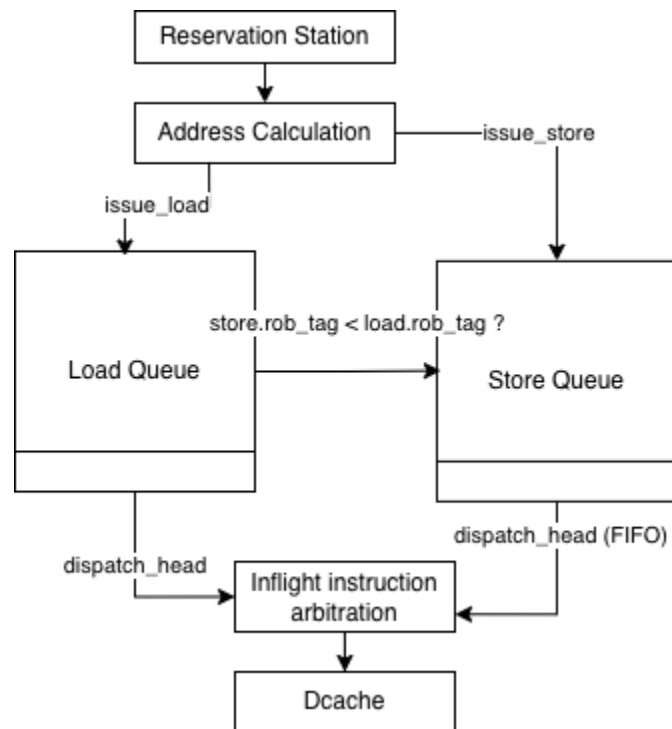


Figure 5: Flowchart depicting split LSQ logic.

The inflight transaction is fully snapshotted and sent to the D-cache interface, allowing the queues themselves to remain stable during cache latency. To support precise recovery, the design

also uses an epoch counter to tag inflight requests so that responses arriving after a flush are deterministically filtered without stalling the cache. This is not necessary but I had it implemented for cleaner flush logic in the queues. Load responses are post-processed for sign extension, and all results and metadata are then broadcast onto the CDB with full architectural information. Stores, in contrast, first broadcast readiness on the CDB, but still retire strictly in order at the head of the Store Queue once committed and acknowledged by memory.

	IPC Recorded Pre-Split LSQ	IPC Recorded Post-Split LSQ
Merge sort	0.4148	0.436554
Aes sha	0.3503	0.391045
FFT	0.4170	0.4848
Compression	0.4745	0.525771
Coremark	0.412468	0.43875

Figure 6: Table showing IPC improvements post split LSQ.

As shown in the previous diagram, the split LSQ significantly improved IPC by reducing memory-induced latency. Prior to this design, loads and stores frequently blocked at the memory functional unit because both operations were effectively serialized through a shared issue path, even when no true ordering or address dependency existed. In particular, unresolved or uncommitted stores would conservatively prevent younger loads from issuing, forcing the pipeline to stall until store data and commit status became available. This behavior limited memory-level parallelism and caused backpressure to propagate into the reservation stations and dispatch stage and loads ended up taking a very large number of cycles before commit.

Dcache Stride Prefetcher

Our data cache stride prefetcher kept a register with the previous addresses that were sent to memory. It tries to find a pattern and fetch the next value in the pattern. However, it catches if the

calculated stride doesn't seem right (if its magnitude is really large), and adjusts to prefetch the next-line instead. This was quite effective because most of the test programs involved some kind of array-like data access.

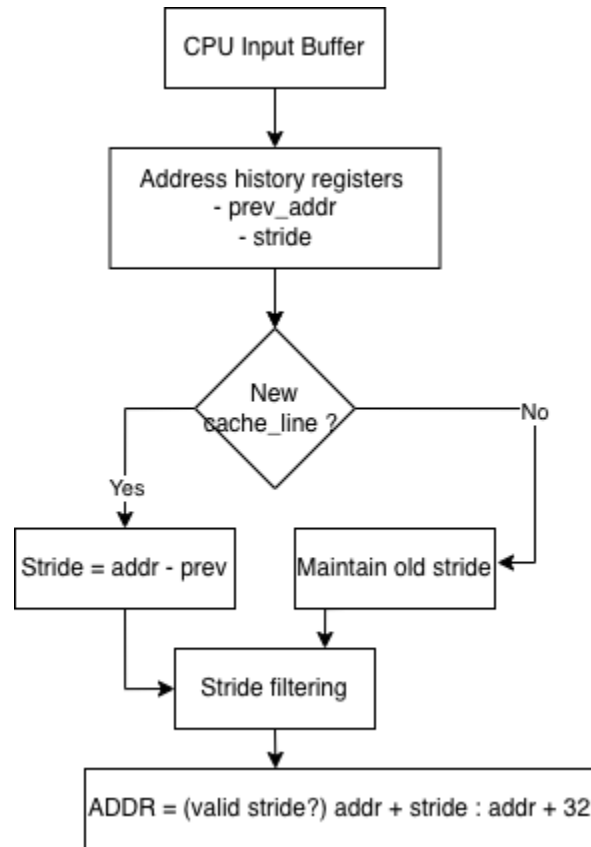


Figure 7: *Stride prefetcher dataflow diagram*

The data wasn't placed directly in cache, but rather kept in a separate buffer in the cacheline adapter. If the data cache went to fetch that address from memory, it would get a response immediately, and would then place it in the cache. This was to avoid overloading our cache with unused data, potentially slowing down our CPU.

Icache next-line prefetcher

The instruction cache next-line prefetcher operates in a very similar way to the dcache prefetcher. This time, it strictly prefetches the next-line (fetch address + 32). The main difference is that this prefetcher forwards the data straight into the fetch stage, so cold-start fetches have

zero latency. The forwarded data still gets placed into the cache, and initiates another next-line prefetch once that happens.

Overall, when the fetch stage jumps from one cacheline to the next (assuming no control hazards), there is at most 3 cycles of latency at 100 MHz. This is vastly different from the original timing, which had about 10 cycles of latency between cachelines.

Return Address Stack

The return address stack was able to highly optimize RISC-V JALR instructions. On call instructions, we push the return address to the RAS on commit. On ret instructions, we pop from the RAS on fetch. Unfortunately, that means some pop instructions don't get their data in time. In this case, we simply stall the fetch stage until either the call instruction commits or the ret instruction passes through the functional unit. We also keep a "committed" version of the RAS, which pushes and pops on commit only. On flushes/mispredictions, the RAS is reset to the committed version, since speculative call/ret instructions might have been performed.

If the data is available on fetch, a ret instruction can pass through the fetch stage and tell us the next PC to jump to with zero latency. Our ROB is still able to determine RAS mispredictions, so we're safe from that. Overall, JALR instructions were optimized to see as few pipeline flushes as possible.

<i>rd is x1/x5</i>	<i>rs1 is x1/x5</i>	<i>rd=rs1</i>	RAS action
No	No	—	None
No	Yes	—	Pop
Yes	No	—	Push
Yes	Yes	No	Pop, then push
Yes	Yes	Yes	Push

Figure 8 : RAS behaviour

Age-ordered issue

For age-ordered issue, we chose to calculate order in the fetch stage and pass it into our reservation station entries. The reservation stations would find the lowest-order ready instruction to issue. This actually had a decent improvement on our IPC, ensuring instructions don't get starved. This was eventually removed due to area and critical path problems. If we could go back and do this again, we would determine age by comparing ROB head index to each instruction's ROB index. This was used in the split LSQ to determine age of load/store instructions.

Conclusion

The objective of this project was to design and implement a fully functional out-of-order RISC-V processor while exploring the processor tradeoffs required to achieve high performance within area and power constraints. Through incremental development across three checkpoints, we successfully built a Tomasulo-style core supporting integer execution, memory operations, and precise in-order retirement using a reorder buffer. Advanced features such as branch prediction, prefetching, a split load-store queue, and return address optimization were incorporated to reduce pipeline stalls and improve overall instruction throughput.

Several novel aspects emerged throughout the design process, including a hybrid register renaming approach, commit-based predictor training to preserve correctness, and a split LSQ that enabled aggressive memory-level parallelism in a student-scale processor. While our final PD⁴ benchmark did not meet the requirements set by the course staff, this limitation highlighted the importance of careful pipeline partitioning and critical path design. Overall, this project provided valuable insight into the complexity of modern processor design and demonstrated how architectural decisions directly impact performance, scalability, and correctness.